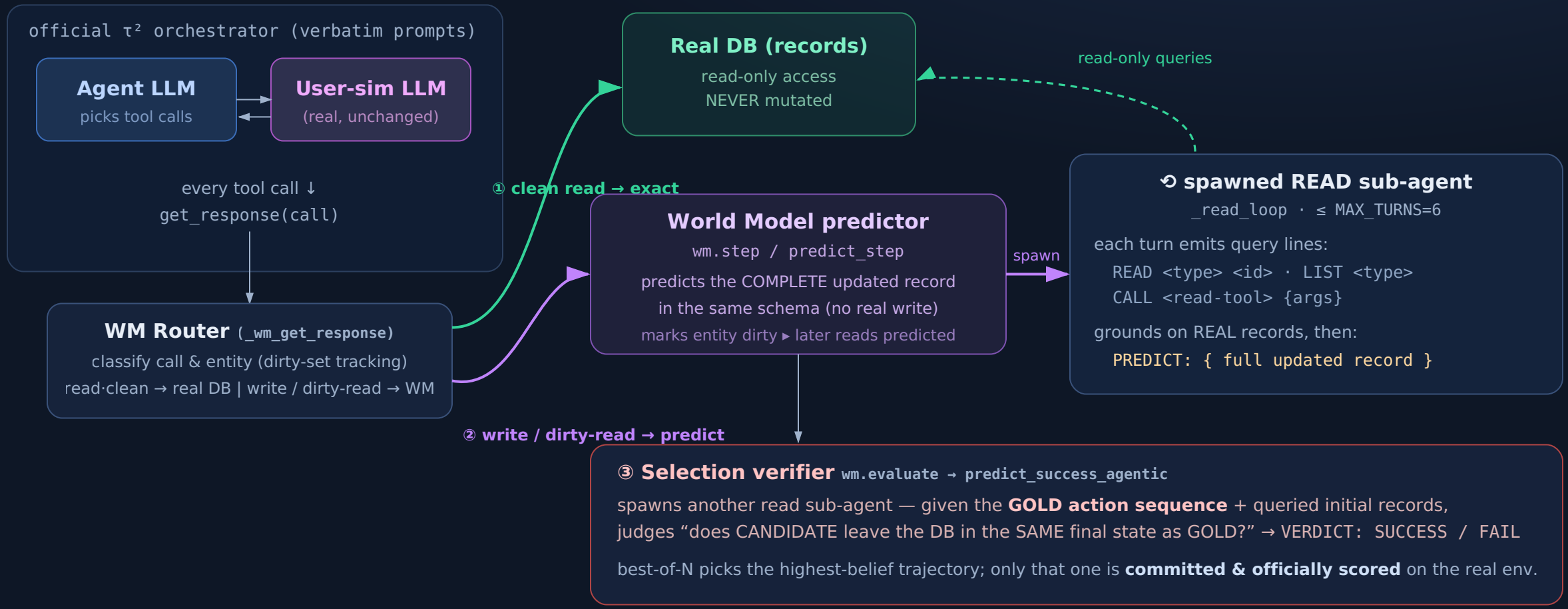
Read-Grounded World Model — how the agent & WM interact “Ours”

Official τ² orchestrator runs unchanged; the single tool-execution chokepoint `environment.get_response` is rerouted through the world model. The real DB is never mutated — only the finally-selected trajectory is committed.



Two failure surfaces: **ACTING** (the agent acts on WM-predicted observations, ①②) and **SELECTING** (the verifier ranks on its own beliefs, ③). The next pages dissect six failures and ask: *did the WM actually have enough information to get it right?*

How to read the next 6 slides

Each slide shows one task as an abstracted trajectory: reads are collapsed, writes highlighted. **b2** = the *same* gpt-4.1 agent acting on the *real* env (oracle-selected) — it succeeds, so its run is the de-facto reference. **ours** = agent acting inside the WM + WM verifier.

ATTRIBUTION BUCKETS

- **read-grounding limit** the WM only knows what was read; the real env validates globally. **Not fixable** without more reads.
- **agent-intrinsic** the decisive error is the agent's choice; a perfect WM would **not** change the outcome.
- **WM error (info present)** the WM/verifier **had** the grounding (incl. gold actions) yet answered wrong.
- **verifier false-positive** broken trajectory stamped **SUCCESS**.

THE AGGREGATE GAP (SUCCESS RATE)

N	airline b2 / ours	retail b2 / ours
1	0.54 / **0.40**	0.67 / **0.62**
8	0.76 / 0.68	0.93 / 0.83
16	0.78 / **0.66** ↓	0.97 / 0.86

Acting gap at N=1 (no selection): airline −0.14, retail −0.04. Verifier false-positives grow 7→11 (airline) & 6→11 (retail) as N→16 — driving the airline N=16 regression.

Honest headline: of the six, most are **read-grounding limits** or **agent-intrinsic** — a better WM *predictor* would not fix them. The clearest "had-enough-info" errors are on the **verifier** side.

Retail task 21 — WM can’t confirm an item it never read

read-grounding limit

OURS — ACTING IN THE WM

```
find_user · get_order ×3 · get_item(9791469541) · get_item(1340995114)
get_product_details(1656367028 = Keyboard)
— Running-Shoes product 6938111410 NEVER read —
modify_pending_order_items(9791469541,1340995114 → 4107812777,1421289881)
WM → “item(s) not found”
modify… (retry) → same error
transfer_to_human “system returns ‘item(s) not found’ error”
```

B2 — SAME AGENT, REAL ENV ✓

```
find_user · get_order ×3
get_product_details(1656367028)
modify_pending_order_items(… → 4107812777, 1421289881) ✓
get_user
```

Verified in tau2 data: `4107812777` ∈ Running Shoes (6938111410); `1421289881` ∈ Keyboard (1656367028). b2 also read only the keyboard product — but the **real env validates items globally**.

Error: WM returns “item(s) not found” for the new shoe variant → agent retries, then escalates to a human. Trajectory ends with no valid modify.

Did the WM have enough info? NO. The Running-Shoes product was never read, so the WM legitimately cannot confirm item 4107812777. Its error is *faithful to its incomplete reads* — an inherent read-grounding limit, not a fabrication.

Airline task 32 — agent gives up; verifier passes a write-less run

agent · verifier FP

OURS

```
get_user(ivan_rossi_8555)
get_reservation ×4 (OWZ4XL, F3IQK1, 8WMF4C, L2H5FP)
— STOP —
ZERO write actions
verifier → predicted_success = TRUE
```

B2 — SAME AGENT, REAL ENV ✓

```
get_user · get_reservation ×4
search_direct_flight(EWR→LAX, 5-21)
update_reservation_flights(OWZ4XL, HAT202+HAT232) ✓
update_reservation_flights(OWZ4XL, HAT041) ✓
```

Error: the agent stops after reading and never rebooks — and the verifier still rates this empty-of-writes trajectory as a success.

Acting = agent-intrinsic: `search_direct_flight` is a read that passes through to the real env — the info was one call away; the agent simply gave up. **Selecting =** genuine WM error: the verifier *is given the gold actions* (which require `update_reservation_flights`) yet passed a candidate with zero writes.

Airline task 17 — no write-confirmation → retry loop, missed step

partial WM · verifier FP

OURS

```
get_user · get_reservation ×4 (incl. FQ8APE ✓)
update_reservation_flights(FQ8APE)
update_reservation_passengers(FQ8APE)
update_reservation_passengers(FQ8APE) ·dup·
update_reservation_passengers(FQ8APE) ·dup·
missing update_reservation_baggages
verifier → predicted_success = TRUE
```

B2 — SAME AGENT, REAL ENV ✓

```
get_user · get_reservation ×4
search_direct / onestop_flight
update_reservation_flights(FQ8APE) ✓
update_reservation_passengers(FQ8APE) ✓
update_reservation_baggages(FQ8APE, total=3) ✓
```

Error: the passenger update is re-issued 3× and the required baggage write is dropped (turn budget burned). Verifier marks it success anyway.

Partly fixable WM: reservation FQ8APE *was* read, so the WM **had** the record to reproduce a clean confirmation and avoid the retry loop — a real (if unprovable-from-logs) acting error. The dropped `update_reservation_baggages` then slips past the verifier even though gold contains it → verifier FP.

Retail task 36 — wrong variant choice on equal grounding

agent-intrinsic

OURS

`find_user · get_order(#W9348897) · get_product ×5 (same 5 as b2)`

`modify_items(… → also swaps 9879255677, 9851293632) should stay`

`cancel_pending_order(#W9348897, “ordered by mistake”)`

B2 — SAME AGENT, REAL ENV ✓

`find_user · get_order · get_product ×5`

`modify_items(keep 9879255677 & 9851293632) ✓`

`modify_items(corrective, 3 items) ✓`

Error: ours changes two items the user wanted kept, then “fixes” it by cancelling the whole order. b2 instead issues a corrective second modify.

Did the WM have enough info? Yes — and it’s irrelevant. ours and b2 read the **identical 5 products**; the wrong `new_item_ids` and the cancel are the agent’s reasoning on equal grounding. A perfect WM predictor would not change this outcome → agent-intrinsic.

Retail task 109 — value comes from dialogue, not a record

agent / dialogue

OURS

```
find_user · get_order(#W1603792) · get_order(#W1092119)
modify_pending_order_address(#W1092119, “760 Elm Ave / 77034”)
modify_user_address(“760 Elm Ave / 77034”)
exchange_delivered_order_items(6501071631 → 4913411651) wrong tool+item
```

B2 — SAME AGENT, REAL ENV ✓

```
find_user · get_order ×2
modify_pending_order_address(#W1603792, “592 Elm Ave / 77242”) ✓
modify_user_address(“592 Elm Ave / 77242”) ✓
modify_pending_order_items(6501071631 → 2106335193) ✓
```

Error: wrong order, wrong street/suite/zip, and wrong final tool + replacement item.

Did the WM have enough info? No grounding to give. The new address comes from the **user dialogue**, not any DB read — the WM cannot “correct” it, and the two runs likely had divergent user turns. The wrong *order* choice (both were read) is the agent’s. → dialogue divergence / agent, not a WM-prediction bug.

Airline task 43 — forbidden action the agent chose

agent decision

OURS

`get_user · get_reservation ×6 (incl. 9HBUV8 ✓)`
`cancel_reservation(9HBUV8)`
`cancel_reservation(9HBUV8) ·dup·`
`transfer_to_human` “system returns ‘Reservation not found’”

B2 — SAME AGENT, REAL ENV ✓

`get_user · get_reservation ×6 → STOP, no writes ✓ (gold = decline, don’t cancel)`

Error: commits a cancel the task forbids, twice, then escalates.

Even a perfect WM wouldn’t fix it. Gold is “don’t cancel” — the agent *chose* to call cancel. The WM’s odd “not found” (it read 9HBUV8 fine) adds noise but isn’t the decisive cause → agent.

SO… DOES THE WM HAVE ENOUGH INFO TO GET IT RIGHT?

task	locus	WM had info?
21	read-grounding	No (unread product)
32	agent + verifier	verifier: yes
17	WM + verifier	partly yes
36	agent	yes, irrelevant
109	dialogue	no grounding
43	agent	N/A

Takeaway: a better WM *predictor* fixes few of these. Real levers: (1) make the **agent read more** before writing (helps 21), (2) a **stricter verifier** that checks gold-action coverage — would catch the clear FPs (32, 17).

acting=agent · + corrected attribution summary